OOP Concepts Study Guide

Encapsulation

What is Encapsulation?

Encapsulation is one of the fundamental principles of Object-Oriented Programming (OOP). It refers to the bundling of data (attributes) and methods (functions) that operate on that data into a single unit called a class. Encapsulation also involves restricting direct access to some of an object's components, which is a means of preventing accidental interference and misuse.

Key Benefits of Encapsulation

- **Data Protection**: Prevents unauthorized access to data
- **Code Maintainability**: Changes to internal implementation don't affect external code
- **Modularity**: Code is organized into discrete, manageable units
- **Data Validation**: Allows controlled access through getter and setter methods

Access Modifiers in Java

Java provides four access modifiers to implement encapsulation:

- **private**: Accessible only within the same class
- **default** (no modifier): Accessible within the same package
- **protected**: Accessible within the same package and subclasses
- **public**: Accessible from anywhere

Example: Basic Encapsulation

java

```java
public class Student {
    // Private data members (encapsulated)
    private String name;
    private int age;
    private double gpa;

    // Constructor
    public Student(String name, int age, double gpa) {
        this.name = name;
        this.age = age;
        this.gpa = gpa;
    }

    // Getter methods (public access to private data)
    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public double getGpa() {
        return gpa;
    }

    // Setter methods (controlled modification of private data)
    public void setName(String name) {
        if (name != null && !name.trim().isEmpty()) {
            this.name = name;
        }
    }

    public void setAge(int age) {
        if (age > 0 && age < 150) {
            this.age = age;
        }
    }

    public void setGpa(double gpa) {
        if (gpa >= 0.0 && gpa <= 4.0) {
            this.gpa = gpa;
        }
    }
}
```

Using the Encapsulated Class

```java
public class Main {
    public static void main(String[] args) {
        Student student = new Student("Alice", 20, 3.5);

        // Accessing data through getter methods
        System.out.println("Name: " + student.getName());
        System.out.println("Age: " + student.getAge());
        System.out.println("GPA: " + student.getGpa());

        // Modifying data through setter methods
        student.setAge(21);
        student.setGpa(3.7);

        // This would cause a compilation error (good!)
        // student.name = "Bob"; // Cannot access private field
    }
}
```

Advanced Encapsulation Example

java

```java
public class BankAccount {
    private String accountNumber;
    private double balance;
    private String accountHolderName;

    public BankAccount(String accountNumber, String accountHolderName) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = 0.0;
    }

    // Method to deposit money
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: $" + amount);
        } else {
            System.out.println("Invalid deposit amount");
        }
    }

    // Method to withdraw money
    public boolean withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: $" + amount);
            return true;
        } else {
            System.out.println("Insufficient funds or invalid amount");
            return false;
        }
    }

    // Getter for balance (read-only access)
    public double getBalance() {
        return balance;
    }

    // Getter for account number (read-only access)
    public String getAccountNumber() {
        return accountNumber;
    }

    // Getter for account holder name
    public String getAccountHolderName() {
        return accountHolderName;
```

```
    }
}
```

Inheritance

What is Inheritance?

Inheritance is a mechanism in OOP that allows a new class (called a subclass or child class) to inherit properties and methods from an existing class (called a superclass or parent class). This promotes code reusability and establishes a relationship between classes.

Key Benefits of Inheritance

- **Code Reusability**: Avoid writing duplicate code
- **Method Overriding**: Customize behavior in subclasses
- **Polymorphism**: Treat objects of different classes uniformly
- **Hierarchical Classification**: Model real-world relationships

Types of Inheritance

Java supports several types of inheritance:

- **Single Inheritance**: One class inherits from one superclass
- **Multilevel Inheritance**: A class inherits from another class, which inherits from another class
- **Hierarchical Inheritance**: Multiple classes inherit from one superclass

Note: Java does not support multiple inheritance of classes but supports it through interfaces.

Basic Inheritance Example

java

```java
// Parent class (Superclass)
public class Animal {
    protected String name;
    protected int age;

    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void eat() {
        System.out.println(name + " is eating");
    }

    public void sleep() {
        System.out.println(name + " is sleeping");
    }

    public void makeSound() {
        System.out.println(name + " makes a sound");
    }

    // Getters
    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }
}

// Child class (Subclass)
public class Dog extends Animal {
    private String breed;

    public Dog(String name, int age, String breed) {
        super(name, age); // Call parent constructor
        this.breed = breed;
    }

    // Override parent method
    @Override
    public void makeSound() {
        System.out.println(name + " barks: Woof! Woof!");
    }
```

```java
    // New method specific to Dog
    public void wagTail() {
        System.out.println(name + " is wagging tail");
    }

    public String getBreed() {
        return breed;
    }
}
```

Using Inheritance

```java
public class Main {
    public static void main(String[] args) {
        // Create parent class object
        Animal animal = new Animal("Generic Animal", 5);
        animal.eat();
        animal.makeSound();

        System.out.println();

        // Create child class object
        Dog dog = new Dog("Buddy", 3, "Golden Retriever");
        dog.eat();           // Inherited method
        dog.sleep();         // Inherited method
        dog.makeSound();     // Overridden method
        dog.wagTail();       // Dog-specific method

        System.out.println("Dog's breed: " + dog.getBreed());
    }
}
```

Method Overriding

Method overriding allows a subclass to provide a specific implementation of a method that is already defined in its parent class.

```java
public class Cat extends Animal {
    private boolean isIndoor;

    public Cat(String name, int age, boolean isIndoor) {
        super(name, age);
        this.isIndoor = isIndoor;
    }

    @Override
    public void makeSound() {
        System.out.println(name + " meows: Meow! Meow!");
    }

    @Override
    public void sleep() {
        System.out.println(name + " sleeps 16 hours a day");
    }

    public void climb() {
        System.out.println(name + " is climbing");
    }

    public boolean isIndoor() {
        return isIndoor;
    }
}
```

Multilevel Inheritance Example

java

```java
// Base class
public class Vehicle {
    protected String brand;
    protected int year;

    public Vehicle(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }

    public void start() {
        System.out.println(brand + " vehicle is starting");
    }

    public void stop() {
        System.out.println(brand + " vehicle is stopping");
    }
}

// Intermediate class
public class Car extends Vehicle {
    protected int numberOfDoors;

    public Car(String brand, int year, int numberOfDoors) {
        super(brand, year);
        this.numberOfDoors = numberOfDoors;
    }

    public void honk() {
        System.out.println(brand + " car is honking");
    }

    @Override
    public void start() {
        System.out.println(brand + " car engine is starting");
    }
}

// Leaf class
public class SportsCar extends Car {
    private int topSpeed;

    public SportsCar(String brand, int year, int numberOfDoors, int topSpeed) {
        super(brand, year, numberOfDoors);
        this.topSpeed = topSpeed;
    }
```

```java
    public void turboBoost() {
        System.out.println(brand + " sports car activating turbo boost!");
    }

    @Override
    public void start() {
        System.out.println(brand + " sports car roaring to life!");
    }

    public int getTopSpeed() {
        return topSpeed;
    }
}
```

The `super` Keyword

The `super` keyword is used to:

- Call the parent class constructor
- Access parent class methods
- Access parent class variables (when they have the same name in child class)

```java
public class Employee {
    protected String name;
    protected double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public void displayInfo() {
        System.out.println("Employee: " + name + ", Salary: $" + salary);
    }
}

public class Manager extends Employee {
    private int teamSize;

    public Manager(String name, double salary, int teamSize) {
        super(name, salary); // Call parent constructor
        this.teamSize = teamSize;
    }

    @Override
    public void displayInfo() {
        super.displayInfo(); // Call parent method
        System.out.println("Team Size: " + teamSize);
    }

    public void conductMeeting() {
        System.out.println(super.name + " is conducting a meeting");
    }
}
```

Quiz Questions

Question 1: What is the primary purpose of encapsulation in Java? a) To make all variables public b) To bundle data and methods together while restricting access to internal details c) To create multiple classes d) To inherit from parent classes

Correct Answer: b

Question 2: Which access modifier makes a field accessible only within the same class? a) public b) protected c) private d) default

Correct Answer: c

Question 3: What keyword is used in Java to establish inheritance between classes? a) implements b) extends c) inherits d) super

Correct Answer: b

Question 4: What happens when a child class overrides a method from its parent class? a) The parent method is deleted b) Both methods are called simultaneously c) The child class method is used instead of the parent method when called on child objects d) A compilation error occurs

Correct Answer: c

Question 5: What is the correct way to call a parent class constructor from a child class constructor? a) parent() b) super() c) this() d) base()

Correct Answer: b

Practice Coding Questions

Question 1: Create a `Person` class with private fields for `name`, `age`, and `email`. Implement proper encapsulation with getter and setter methods. The setter methods should include validation (age should be positive, email should contain '@' symbol, name should not be empty).

Question 2: Design a class hierarchy for a library system. Create a base class `LibraryItem` with common properties like `title`, `author`, and `isbn`. Then create two subclasses: `Book` (with additional property `numberOfPages`) and `Magazine` (with additional property `issueNumber`). Implement method overriding for a `getDescription()` method in each class.

Question 3: Create a `Shape` class with a method `calculateArea()`. Create two subclasses `Rectangle` and `Circle` that inherit from `Shape`. Override the `calculateArea()` method in both subclasses to calculate the area specific to each shape. Include appropriate constructors and encapsulated fields.

Question 4: Design a banking system with a base class `Account` containing `accountNumber`, `balance`, and `accountHolderName`. Create two subclasses: `SavingsAccount` (with interest rate) and `CheckingAccount` (with overdraft limit). Implement proper encapsulation and method overriding for deposit and withdrawal operations.

Question 5: Create a `Vehicle` class and implement multilevel inheritance with `Car` extending `Vehicle`, and `ElectricCar` extending `Car`. Each class should have specific properties and methods. Demonstrate the use of the `super` keyword in constructors and method calls.